RETAILMART V3 · SQL

SQL Self Join & UNION

— Siraj Sir

📱 WhatsApp Announcement

SELF JOIN & UNION

Yesterday: stitching DIFFERENT tables side-by-side. Today: TWO new patterns. (1) SELF JOIN — joining a table to ITSELF, the trick that powers 'mutual friends', 'similar products', 'employee-manager' queries. (2) UNION — stacking results VERTICALLY when you want one feed from multiple sources. Two patterns, both production-essential.

Today's Focus:

- SELF JOIN — same table, two aliases, row-by-row comparison
- Hierarchies, duplicates, pairs — all SELF JOIN territory
- UNION & UNION ALL — combining query results vertically
- When to use UNION vs UNION ALL (performance + intent)

This wraps up the join toolkit. Tomorrow we move into subqueries.
— Siraj Sir
#Day11 #PostgreSQL #SelfJoin #Union

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap	5 mins	FULL OUTER + multi-table JOINs recap, Star vs chain patterns
Part 1: SELF JOIN	55 mins	Same table, two aliases — the pattern, De-duplication with a.id < b.id, Hierarchies, colleague pairs, duplicate detection, 4 practice problems (Easy → Crazy)
Part 2: UNION & UNION ALL	55 mins	UNION — stack and deduplicate, UNION ALL — stack and KEEP duplicates (faster), Column matching rules: count, types, names, 4 practice problems (Easy → Crazy)
Quiz + Summary & Wrap-up	25 mins	Try It Yourself challenges, Quick quiz (5 questions), Common mistakes, Interview bank, Homework + Subqueries Part 2 preview

YOUR SQL JOURNEY

- ✓ Scalar Functions
- ✓ CASE & COALESCE
- ✓ Aggregates & GROUP BY
- ✓ JOINs Part 1
- ✓ JOINs Part 2
- 11 **Self Join & UNION** ← HERE
- 12 Subqueries Part 1
- 13 Subqueries Part 2 + CTEs

RECAP

Memory Check

FULL OUTER JOIN for reconciliation (preserve unmatched from both sides). Multi-table JOINs spanning 4–7 tables — star pattern, chain pattern, mixing INNER and LEFT.

Today: Two new patterns that look very different from JOINs. SELF JOIN compares rows WITHIN the same table. UNION combines results from SEPARATE queries vertically. Together, they handle scenarios JOINs can't.

STORY

Two Patterns from One Reunion

At a class reunion, two questions come up. First: 'Who else from MY school is here?' That means comparing each guest to other guests from the SAME guest list — a SELF JOIN. Same table, two views, row-by-row matching.

Second: 'Combine RSVP responses from email and SMS into one master attendance list.' Two different sources, same column shape (name + email), stacked into one list — that's UNION. Two patterns, two everyday problems.

REVEAL

From Story to SQL!

'Who from MY class is also here' — SELF JOIN on shared attribute
'Find pairs' — SELF JOIN with a.id < b.id de-duplication
'Combine RSVP sources' — UNION ALL (no duplicates expected)
'Unique attendees across sources' — UNION (deduplicates)
Both patterns work on top of everything you already know

STORY

The Mutual Friends Problem

On social media, you and your friend both follow some of the same people. To find mutual follows, the app JOINs the 'follows' table to ITSELF — your follows on the left, your friend's follows on the right, matched by user_id. Same table, two aliases, row-by-row comparison. That's a SELF JOIN.

DEFINITION

SELF JOIN

Technical:
A SELF JOIN joins a table to itself using two different aliases. Row from alias 'a' is compared to row from alias 'b' in the SAME table. Use cases: finding pairs, comparing rows, hierarchy traversal (employee → manager), and duplicate detection.

Layman's:
SELF JOIN = 'Make TWO copies of this table, then JOIN them together to compare rows within the original.' Same data, two views, paired up. The trick: use a.id < b.id to avoid (A,B) AND (B,A) duplicates.

SYNTAX: SELF JOIN

```sql
-- Basic SELF JOIN (find pairs)
SELECT a.col, b.col
FROM table_name a
JOIN table_name b
  ON a.shared_attribute = b.shared_attribute
  AND a.id < b.id;  -- avoid duplicates!

-- a.id < b.id prevents three problems:
-- (Alice, Bob) AND (Bob, Alice) — duplicate pair
-- (Alice, Alice) — self-pairing
-- Row count roughly doubles without it

-- Hierarchy variant (parent-child)
SELECT child.name, parent.name
FROM employees child
LEFT JOIN employees parent
  ON child.manager_id = parent.employee_id;
```

Step-by-Step: How SELF JOIN Works

```
-- employees:
-- emp_id | name    | dept_id
-- -------|---------|--------
-- 1      | Priya   | 10
-- 2      | Rahul   | 10
-- 3      | Amit    | 20

-- SELF JOIN: same dept, a.emp_id < b.emp_id
-- Priya(1) + Rahul(2) dept 10  ✓ PAIR
-- Priya(1) + Amit(3)  dept ≠   ✗ SKIP
-- Rahul(2) + Amit(3)  dept ≠   ✗ SKIP

-- Result: 1 pair (Priya, Rahul) — same dept
-- a.id < b.id avoids (Rahul, Priya) duplicate
```

Easy: Employees Sharing a Department

EASY

SCENARIO: HR wants to see pairs of employees who work in the same department — for cross-training nominations. Each pair should appear only once, not twice.

YOUR TASK: SELF JOIN employees on dept_id. Use e1.employee_id < e2.employee_id to avoid duplicate pairs. Show both names, their shared department, and both roles.

🤔 **Hint:** Joining a table to itself means two aliases — think of them as two 'copies'. Trap: a JOIN on just the shared attribute (dept_id) produces (Alice, Bob) AND (Bob, Alice) AND (Alice, Alice) — you need an extra condition to enforce a one-way pairing.

Answer

✅ ANSWER

```
SELECT
  e1.first_name || ' ' || e1.last_name
    AS employee_1,
  e1.role AS role_1,
  e2.first_name || ' ' || e2.last_name
    AS employee_2,
  e2.role AS role_2,
  d.dept_name
FROM stores.employees e1
JOIN stores.employees e2
  ON e1.dept_id = e2.dept_id
  AND e1.employee_id < e2.employee_id
JOIN core.dim_department d
  ON e1.dept_id = d.dept_id
ORDER BY d.dept_name
LIMIT 15;
```

Medium: Product Pairs in the Same Category

MEDIUM

SCENARIO: The merchandising team wants to power a 'frequently compared' feature. Find pairs of products that share the same category — these are products customers compare against each other.

YOUR TASK: Products don't have a category column directly — go through dim_brand → dim_category. SELF JOIN products via this chain. Show both names, category, and the price difference. Use p1.product_id < p2.product_id for unique pairs.

🤔 **Hint:** Category sits two FK hops from products — both copies of the table need to walk the chain. Trap: doing the category chain ONCE (only for p1) gives wrong results — both p1 AND p2 need their category resolved to compare them.

Answer

✅ ANSWER

```
SELECT
  p1.product_name AS product_1,
  p1.price AS price_1,
  p2.product_name AS product_2,
  p2.price AS price_2,
  cat.category_name,
  ABS(p1.price - p2.price)
    AS price_diff
FROM products.products p1
JOIN core.dim_brand b1
  ON p1.brand_id = b1.brand_id
JOIN products.products p2
  ON p2.product_id > p1.product_id
JOIN core.dim_brand b2
  ON p2.brand_id = b2.brand_id
  AND b2.category_id = b1.category_id
JOIN core.dim_category cat
  ON b1.category_id = cat.category_id
ORDER BY cat.category_name, price_diff
LIMIT 15;
```

Hard: Web Session Page Pairs — User Journey Analysis

HARD

SCENARIO: The Product team wants to map user journeys: which page pairs are visited together within the SAME session? SELF JOIN web_events.page_views on session_id, find pairs where one page was viewed before another, and surface the most common transitions.

YOUR TASK: SELF JOIN web_events.page_views on session_id. Use pv1.view_timestamp < pv2.view_timestamp to enforce chronological order (not just unique pairs). Show session_id, from_page, to_page, and the time gap in seconds.

🤔 **Hint:** Unlike pair-finding (a.id < b.id), JOURNEY analysis cares about CHRONOLOGY — compare timestamps, not IDs. Trap: a user who views the same page twice in a session would produce a self-pair if you forget the inequality; the timestamp comparison handles both dedup AND ordering.

Answer

✅ ANSWER

```
SELECT
  pv1.session_id,
  pv1.page_url AS from_page,
  pv2.page_url AS to_page,
  ROUND(EXTRACT(EPOCH FROM
    pv2.view_timestamp - pv1.view_timestamp))
    AS seconds_between
FROM web_events.page_views pv1
JOIN web_events.page_views pv2
  ON pv1.session_id = pv2.session_id
  AND pv1.view_timestamp
    < pv2.view_timestamp
ORDER BY pv1.session_id,
  pv1.view_timestamp
LIMIT 20;
```

Crazy: Salary Inequity Detection

CRAZY

SCENARIO: The CHRO wants to detect potential pay inequity: find pairs of employees in the same store and same role whose salaries differ by more than 20%. These flagged pairs go to the compensation committee for review.

YOUR TASK: SELF JOIN employees on store_id AND role. Calculate salary difference and percentage gap. Filter pairs where the gap exceeds 20%. Show both names, store, role, and salary details.

🤔 **Hint:** Pay-gap detection layers TWO matching conditions (store + role) plus the dedupe inequality. Trap: dividing the gap by ONE of the two salaries is asymmetric — pairs A,B and B,A would give different percentages; divide by the larger to keep it symmetric.

Answer (1/2)

✅ ANSWER

```
SELECT
  s.store_name,
  e1.role,
  e1.first_name || ' ' || e1.last_name
    AS employee_1,
  e1.salary AS salary_1,
  e2.first_name || ' ' || e2.last_name
    AS employee_2,
  e2.salary AS salary_2,
  ABS(e1.salary - e2.salary)
    AS salary_gap,
  ROUND(ABS(e1.salary - e2.salary)
    * 100.0 / GREATEST(
    e1.salary, e2.salary), 1)
    AS gap_pct
FROM stores.employees e1
JOIN stores.employees e2
  ON e1.store_id = e2.store_id
  AND e1.role = e2.role
  AND e1.employee_id < e2.employee_id
JOIN stores.stores s
  ON e1.store_id = s.store_id
```

Answer (2/2)

✅ ANSWER

```
WHERE ABS(e1.salary - e2.salary)
  * 100.0 / GREATEST(
  e1.salary, e2.salary) > 20
ORDER BY gap_pct DESC
LIMIT 15;
```

KEY TAKEAWAYS

SELF JOIN

SELF JOIN = join a table to itself with two aliases. Use a.id < b.id to avoid duplicate pairs and self-pairing. Use cases: finding colleagues in the same department, products in the same category, same-day orders, salary comparisons, duplicate detection. Always use meaningful aliases (e1/e2, o1/o2, p1/p2).

REAL WORLD

SELF JOIN in Production

SELF JOIN is the secret behind many features you use daily:

- LinkedIn: 'People you may know' = SELF JOIN on company/school
- Instagram: 'Mutual friends' = SELF JOIN on following relationships
- Amazon: 'Frequently bought together' = SELF JOIN on order_items
- Zomato: 'Restaurants similar to X' = SELF JOIN on cuisine/area

Anywhere you see 'similar', 'related', or 'mutual' in a UI — there is probably a SELF JOIN powering it.

COMMON MISTAKE

Forgetting the De-Duplication Condition

The Mistake:
SELF JOIN without a.id < b.id. Result: (Alice, Bob) AND (Bob, Alice) appear as separate rows, plus (Alice, Alice) self-pairs. Row count doubles and every pair is counted twice in aggregates.

The Fix:
Always add AND a.id < b.id (or a.id != b.id if you want both directions but no self-pairs). Use < for unique pairs, != for both directions.

PRO TIP

PRO TIP

SELF JOIN performance: a table with 10,000 rows self-joined produces up to 100 MILLION combinations before filtering. Always add restrictive ON conditions (same dept, same city, same date) to limit the pairs EARLY. The more conditions in ON, the faster the query.

STORY

The Cricket Team Selection

BCCI selects a national team. The list comes from TWO sources: state-level Ranji performers AND IPL standout players. UNION merges both lists into ONE roster — if a player appears in BOTH (Ranji AND IPL star), UNION shows them ONCE. UNION ALL shows them TWICE. UNION = unique roster. UNION ALL = combined attendance with duplicates.

DEFINITION

UNION vs UNION ALL

Technical:
UNION combines results of two SELECTs and removes duplicate rows. UNION ALL combines without removing duplicates. UNION must compare every row to deduplicate — UNION ALL just concatenates. UNION ALL is significantly faster on large datasets.

Layman's:
UNION = 'Merge these two lists and keep only unique entries.' UNION ALL = 'Just stack them — duplicates allowed.' If you KNOW there are no duplicates, ALWAYS prefer UNION ALL for performance.

SYNTAX: UNION and UNION ALL

```sql
-- UNION removes duplicates
SELECT first_name, email
FROM customers.customers
WHERE registration_date > '2024-01-01'
UNION
SELECT first_name, email
FROM stores.employees;

-- UNION ALL keeps duplicates (faster)
SELECT name, source FROM ...
UNION ALL
SELECT name, source FROM ...;

-- ORDER BY goes at the END (final result)
SELECT ... UNION SELECT ... ORDER BY 1;

-- Iron rules:
-- (1) Same number of columns in each SELECT
-- (2) Compatible data types in matching positions
-- (3) Column names come from the FIRST query
```

Step-by-Step: How UNION Works

```
-- Query A:                Query B:
-- name    | source         name    | source
-- --------|-------         --------|-------
-- Priya   | Customer       Rahul   | Employee
-- Rahul   | Customer       Amit    | Employee

-- UNION result (dedup):
-- Priya  | Customer
-- Rahul  | Customer
-- Rahul  | Employee   ← different SOURCE, kept
-- Amit   | Employee
-- Total: 4 rows

-- UNION ALL: same 4 rows (no dedup needed here)
-- Duplicates only happen on IDENTICAL rows
```

Easy: All People Contact List

EASY

SCENARIO: The CFO is sending a Diwali greeting to EVERY person associated with RetailMart — employees and customers. She needs ONE unified contact list with name, email, and a 'person_type' label.

YOUR TASK: UNION ALL employees and customers. Each SELECT has 3 columns: full name, email, and a literal person_type ('Employee' or 'Customer'). Order by person_type, name.

🤔 **Hint:** Two UNION variants exist — one deduplicates, one doesn't. Pick based on whether duplicates are possible. Trap: employee and customer rows can never collide (different tables, different IDs), so deduplication is wasted CPU — the faster variant gives the same result on disjoint sources.

Answer

✅ ANSWER

```
SELECT
  first_name || ' ' || last_name
    AS full_name,
  email,
  'Employee' AS person_type
FROM stores.employees
UNION ALL
SELECT
  first_name || ' ' || last_name
    AS full_name,
  email,
  'Customer' AS person_type
FROM customers.customers
ORDER BY person_type, full_name
LIMIT 25;
```

Medium: Unified Audit Activity Feed

MEDIUM

SCENARIO: The DevOps lead needs a single chronological feed combining application logs and API requests for incident review. Two audit tables, same date column shape, two different event types.

YOUR TASK: UNION ALL audit.application_logs and audit.api_requests. Each SELECT exposes: timestamp, event_type, severity_or_status, message_or_endpoint. Show 30 most recent events.

🤔 **Hint:** Two different event sources need to align on the SAME column shape — pick column names from the FIRST query. Trap: api_requests has a numeric status_code while application_logs has a 'level' string — cast both to a common type or use ::TEXT to make them compatible.

Answer

✅ ANSWER

```
SELECT
  timestamp,
  'AppLog' AS event_type,
  level AS severity,
  message AS detail
FROM audit.application_logs
UNION ALL
SELECT
  timestamp,
  'ApiRequest' AS event_type,
  status_code::TEXT AS severity,
  endpoint AS detail
FROM audit.api_requests
ORDER BY timestamp DESC
LIMIT 30;
```

Hard: Top Performers Leaderboard

HARD

SCENARIO: The VP of Sales wants a 'Top Performers' leaderboard combining the top 10 spending customers AND the top 10 highest-paid employees in ONE ranking — by monetary score.

YOUR TASK: UNION ALL the top 10 customers (by total spend) and top 10 employees (by salary). Show name, score, category. Sort by score DESC.

🤔 **Hint:** Top-N per source means each side needs its own LIMIT BEFORE the union — ORDER BY at the top level applies to the COMBINED result, not each source. Trap: customer 'score' (a spend SUM) and employee 'score' (a salary integer) need compatible types — cast one side if PostgreSQL complains.

Answer (1/2)

✅ ANSWER

```
SELECT * FROM (
  SELECT
    c.first_name || ' ' || c.last_name
      AS name,
    SUM(o.net_total) AS score,
    'Top Customer' AS category
  FROM customers.customers c
  JOIN sales.orders o
    ON c.customer_id = o.cust_id
  GROUP BY c.customer_id,
    c.first_name, c.last_name
  ORDER BY score DESC LIMIT 10
) t
UNION ALL
SELECT * FROM (
  SELECT
    first_name || ' ' || last_name
      AS name,
    salary::numeric AS score,
    'Top Employee' AS category
  FROM stores.employees
  ORDER BY salary DESC LIMIT 10
```

Answer (2/2)

✅ ANSWER

```
) e
ORDER BY score DESC;
```

Crazy: Cross-Schema Daily Activity Feed

CRAZY

SCENARIO: The Head of Operations wants a chronological feed of recent events from THREE sources: new orders, new tickets, new returns. Each row has a timestamp, type, customer name, and amount. Sorted by time.

YOUR TASK: UNION ALL three queries: orders (last 7 days), support tickets (last 7 days), returns (last 7 days). Each row has the same columns. Sort by event_time DESC.

🤔 **Hint:** Three sources with three different shapes need to align on a common column list. Trap: some sources have dates, others have timestamps — explicit casts (::timestamp) on every source remove ambiguity and prevent silent type mismatches.

Answer (1/3)

✅ ANSWER

```
-- Five sources, one unified customer-
-- activity feed: orders, support tickets,
-- call_center calls, loyalty redemptions,
-- and web page views.

SELECT
  o.order_date::timestamp AS event_time,
  'Order' AS event_type,
  c.first_name || ' ' || c.last_name
    AS customer,
  o.net_total::numeric AS amount
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
WHERE o.order_date >= CURRENT_DATE - 7

UNION ALL

SELECT
  t.created_date AS event_time,
  'Ticket' AS event_type,
  c.first_name || ' ' || c.last_name
```

Answer (2/3)

✅ ANSWER

```
    AS customer,
  0::numeric AS amount
FROM support.tickets t
JOIN customers.customers c
  ON t.customer_id = c.customer_id
WHERE t.created_date >= CURRENT_DATE - 7

UNION ALL

SELECT
  cc.call_start_time AS event_time,
  'Call' AS event_type,
  c.first_name || ' ' || c.last_name
    AS customer,
  cc.call_duration_seconds::numeric
    AS amount
FROM call_center.calls cc
JOIN customers.customers c
  ON cc.customer_id = c.customer_id
WHERE cc.call_start_time
  >= CURRENT_DATE - 7
```

Answer (3/3)

✅ ANSWER

```
UNION ALL

SELECT
  red.redemption_date::timestamp
    AS event_time,
  'Loyalty Redemption' AS event_type,
  c.first_name || ' ' || c.last_name
    AS customer,
  red.points_redeemed::numeric AS amount
FROM loyalty.redemptions red
JOIN customers.customers c
  ON red.customer_id = c.customer_id
WHERE red.redemption_date
  >= CURRENT_DATE - 7

ORDER BY event_time DESC LIMIT 30;
```

KEY TAKEAWAYS

UNION and UNION ALL

UNION merges two result sets and removes duplicates. UNION ALL keeps duplicates and is FASTER. Use UNION ALL when you KNOW there are no duplicates. Both require: SAME number of columns, compatible types, names from the FIRST query. ORDER BY goes at the very end (applies to final combined result).

REAL WORLD

UNION in Production

Where UNION powers real applications:

- Gmail: All inbox + sent + drafts in 'All Mail' = UNION ALL
- LinkedIn: Activity feed combining posts + comments + shares
- PayTM: Transaction history merging UPI + cards + wallet
- Zomato: Search results combining dishes + restaurants + cuisines

Any feature showing 'all your X across categories' is likely a UNION ALL behind the scenes.

COMMON MISTAKE

UNION vs UNION ALL Performance

The Mistake:
Using UNION when you know there are no duplicates. UNION sorts and deduplicates — expensive on large datasets. On 10 million rows, UNION can be 10× slower than UNION ALL.

The Fix:
Default to UNION ALL. Only use UNION when deduplication is genuinely needed AND you can't guarantee uniqueness from query design. When combining different sources (employees + customers), use UNION ALL — they can't overlap.

COMMON MISTAKE

ORDER BY in Wrong Place

The Mistake:

SELECT * FROM a ORDER BY name UNION SELECT * FROM b — ERROR! ORDER BY can't appear before UNION.

The Fix:

ORDER BY goes ONCE at the very end and applies to the final combined result. If you need to sort each side individually (for top-N), wrap each SELECT in a subquery with its own ORDER BY + LIMIT.

STORY

Try It Live!

Run each query. Swap UNION for UNION ALL and check row counts. Modify the SELF JOIN conditions. Understanding comes from experimentation.

Challenge 1: SELF JOIN — Stores in Same City

```
-- Find store pairs in the same city
SELECT s1.store_name AS store_1,
  s2.store_name AS store_2, s1.city
FROM stores.stores s1
JOIN stores.stores s2
  ON s1.city = s2.city
  AND s1.store_id < s2.store_id
ORDER BY s1.city LIMIT 15;
```

Challenge 2: UNION ALL — Audit Feed

```
-- App logs + API requests combined
SELECT timestamp, 'AppLog' AS source,
  level AS detail
FROM audit.application_logs
UNION ALL
SELECT timestamp, 'ApiRequest',
  status_code::TEXT
FROM audit.api_requests
ORDER BY timestamp DESC LIMIT 20;
```

Challenge 3: SELF JOIN — Same Price Products

```
-- Find product pairs with identical prices
SELECT p1.product_name AS product_1,
  p2.product_name AS product_2, p1.price
FROM products.products p1
JOIN products.products p2
  ON p1.price = p2.price
  AND p1.product_id < p2.product_id
ORDER BY p1.price DESC LIMIT 15;
```

Challenge 4: UNION vs UNION ALL Demo

```
-- Same query, two variants — compare counts

-- Variant A: UNION (dedup)
SELECT email FROM customers.customers
UNION
SELECT email FROM stores.employees;

-- Variant B: UNION ALL (no dedup)
SELECT email FROM customers.customers
UNION ALL
SELECT email FROM stores.employees;

-- YOUR TURN: Compare COUNT(*) of each
```

COMMON MISTAKE

SELF JOIN Without Meaningful ON Condition

The Mistake:
FROM employees e1 JOIN employees e2 ON e1.employee_id != e2.employee_id — this pairs EVERY employee with every other employee. 5000 × 4999 = 25 million rows.

The Fix:
Always add a meaningful matching attribute: same dept, same store, same date. Then add e1.id < e2.id for unique pairs. The ON clause should be restrictive, not permissive.

COMMON MISTAKE

Using UNION When UNION ALL Suffices

The Mistake:
UNION when you KNOW there are no duplicates. Example: combining employee emails and customer emails — these tables can't share IDs. UNION wastefully sorts and deduplicates.

The Fix:
Default to UNION ALL. Only use UNION when deduplication is genuinely needed. Performance difference is huge on large datasets.

COMMON MISTAKE

Column Type Mismatch

The Mistake:
SELECT product_id, price UNION SELECT customer_id, first_name — column 2 mismatch (NUMERIC vs VARCHAR). PostgreSQL throws a type error.

The Fix:
Match types in matching positions. Use explicit casts: amount::TEXT or status_code::TEXT to align types. Or restructure the query to make the comparison meaningful.

COMMON MISTAKE

ORDER BY Goes Last (UNION)

The Mistake:

SELECT * FROM a ORDER BY name UNION SELECT * FROM b — ERROR. ORDER BY cannot appear before UNION.

The Fix:

ORDER BY goes ONCE at the very end. To sort each side individually (for top-N per source), wrap each SELECT in a subquery with its own ORDER BY + LIMIT.

INTERVIEW

How Does SELF JOIN Work?

Q: 'Explain SELF JOIN with an example.'

A: SELF JOIN joins a table to itself using two aliases. Example: finding employee pairs in the same department — JOIN employees e1 WITH employees e2 ON e1.dept_id = e2.dept_id AND e1.id < e2.id. The < prevents duplicate pairs. Use cases: same-day orders, stores in same city, salary comparisons, duplicate detection.

INTERVIEW

SELF JOIN for Duplicates

Q: 'How do you find duplicate records in a table?'

A: SELF JOIN on the columns that should be unique: JOIN employees e1, e2 ON e1.email = e2.email AND e1.employee_id < e2.employee_id. Rows that come back mean two different IDs share the same email — a duplicate. The < condition ensures each duplicate pair appears once.

INTERVIEW

UNION vs UNION ALL

Q: 'When would you use UNION ALL instead of UNION?'

A: UNION ALL when you KNOW the two sets can't have duplicates, OR when you actually want to count duplicates. Examples: combining employees and customers (disjoint sets), or combining today's orders with yesterday's orders to count total volume. UNION ALL is significantly faster — it skips the deduplication step.

INTERVIEW

UNION Requirements

Q: 'What are the rules for using UNION?'

A: Three rules. (1) Same number of columns in both SELECTs. (2) Compatible data types in matching POSITIONS (not by name). (3) Column NAMES come from the FIRST query — second query's names are ignored. Also: ORDER BY only at the very end of the combined query.

INTERVIEW

SELF JOIN Performance

Q: 'Why can SELF JOIN be slow on large tables?'

A: SELF JOIN can produce up to N^2 combinations before filtering — for a 10,000-row table that's 100 million pairs. Always add restrictive ON conditions (same dept, same city, same date) plus an inequality (a.id < b.id) to reduce the pair space early. Make sure the matching columns are indexed.

INTERVIEW

UNION Column-Name Behavior

Q: 'I named a column 'customer_name' in the first SELECT and 'name' in the second. What does the result column get called?'

A: 'customer_name' — UNION always uses names from the FIRST query. The second query's column names are ignored, even if they match in some positions. This is why putting the 'canonical' query first matters.

INTERVIEW

Real Interview: Hierarchies Without manager_id

Q: 'You want a SELF JOIN for an employee hierarchy, but the table has no manager_id column. What now?'

A: Adapt to what's available. If there's role-based hierarchy (CEO > Manager > Staff), SELF JOIN on role pairs. If there's store-based hierarchy (Store Manager → Staff), SELF JOIN on store_id with a role filter. If no hierarchy exists, the data simply isn't there — push back to data engineering.

KEY TAKEAWAYS

KEY TAKEAWAYS

SELF JOIN:

1. Same table, two aliases, row-by-row comparison.
2. Always use a.id < b.id for unique pairs.
3. Use cases: colleagues, same-day orders, salary comparison, duplicates.
4. Restrict the ON clause heavily — pair count is N^2 without filters.

UNION / UNION ALL:

5. UNION merges + deduplicates. UNION ALL keeps duplicates (faster).
6. Three rules: same column count, compatible types, names from first query.
7. ORDER BY goes at the very end (applies to the final combined result).
8. Default to UNION ALL — only use UNION when deduplication is necessary.

REAL WORLD

What You Achieved Today

You learned two patterns that handle problems plain JOINs can't. SELF JOIN compares rows within a single table — the engine behind 'similar items', 'mutual connections', and 'duplicate detection'. UNION stacks results vertically — the engine behind unified feeds, top-N leaderboards, and multi-source reports.

This wraps up the JOIN toolkit. You now have INNER, LEFT, RIGHT, FULL OUTER, SELF, plus UNION/UNION ALL. With these, you can combine data in any direction.

Tomorrow: Subqueries Part 1. Putting queries inside queries — scalar subqueries, IN-subqueries, and derived tables in FROM.

SELF JOIN & UNION

```
-- SELF JOIN (pairs)

 FROM table a JOIN table b
   ON a.match = b.match
   AND a.id < b.id  -- unique pairs

-- SELF JOIN (hierarchy)

 FROM table child LEFT JOIN table parent
   ON child.parent_id = parent.id
 -- For employee → manager etc.

-- UNION (dedup)

 SELECT col FROM a
 UNION
 SELECT col FROM b;
 -- Merges + removes duplicates
```

SELF JOIN & UNION

-- UNION ALL (faster)

```
SELECT col FROM a
UNION ALL
SELECT col FROM b;
-- Merges + keeps duplicates
```

-- ORDER BY at End

```
SELECT ... UNION SELECT ...
ORDER BY 1, 2  -- positional
-- Applies to final result
```

-- UNION Iron Rules

1. Same column COUNT
2. Compatible TYPES (by position)
3. Names from FIRST query

CHALLENGE

Subqueries Part 1 Take-Home Challenge (1/2)

Build these queries on RetailMart V3:

Easy (SELF JOIN):

1.: Employees sharing a department — show both names and dept name. Use < for unique pairs.

2.: Stores in the same city — show both store names and city.

CHALLENGE

Subqueries Part 1 Take-Home Challenge (2/2)

Medium (UNION):

3.: UNION ALL employees and customers for a unified contact list with person_type label.

4.: UNION ALL app_logs + api_requests into a single audit feed sorted by timestamp.

Hard:

5.: Product pairs in the same category (SELF JOIN through dim_brand + dim_category).

6.: UNION vs UNION ALL row-count comparison — count both variants of the same query.

Crazy:

7.: Salary inequity detection: employees in same store + role with >20% salary gap. SELF JOIN + percentage math.

Push your SQL file to GitHub!

https://github.com/starlordali4444/postgresql

STORY

Coming Tomorrow

Subqueries — putting one query INSIDE another. Scalar subqueries (return one value, use in WHERE/SELECT). IN-subqueries (filter by a dynamic list). Subqueries in FROM (derived tables for pre-aggregation).

This is where SQL composition really kicks in. Combined with JOINs and set ops, you can express ANY analyst-level business question. earlier sessions are the next big level-up.